Simulate First? What Cheap Rule-Agent Worlds Predict About Teams of Language-Model Agents

Chuntao Dan

October 18, 2026

Abstract

Designing a team of language-model agents means choosing how agents communicate, how many there are, and how they are rewarded, and every choice is expensive to test with real models. We ask whether a cheap simulation can screen these choices first. We built small numpy worlds with simple learning or rule-following agents, tested pre-registered hypotheses in them, and then ran the same designs with Claude Haiku 5.5 teams and, for reward designs, with a 3B open model trained by reinforcement learning. In the simulations all four pre-registered tests were supported on 30 seeds each: a shared blackboard kept output per agent up as teams grew at fixed compute, sped recovery after the environment changed, reduced belief diversity, and made teams more vulnerable to a liar than peer messages did. With language-model teams the picture depends on the task. The simulations ranked harness designs as the LLM teams did on a task board ($\rho = 0.75$) and on a clue-sharing board ($\rho = 0.79$), but not on text foraging ($\rho = 0.02$), where the LLM teams herded. For RL reward designs, simple numpy learners trained under the same rewards reproduced 7 of 12 effect signs of completed runs and, in a forecast committed before the evaluation, 4 of 8: they predicted that credit with per-turn advantages would beat an imitation start, but missed that pricing a guessing loophole would keep agents sharing. A third forecast, of how a team changes from 8 to 32 agents, got 1 of 2 trends right: pooling falls, but the simulation predicted that a fixed share of liars would do less damage in larger teams, and the opposite happened; tracing both runners showed that the error lay in the simulated clean team, which degraded too fast, not in how its agents handled lies. Simulations are a useful screen for how information flows through a team, and a poor one for what language models learn or how they read text.

1 Introduction

Agent teams are designed by trial: channels (a shared board, peer messages, a coordinator), team size, memory, and incentives. With real models each trial costs API calls or GPU hours; with reinforcement learning, a single run can take hours and, as we show in a companion paper, results vary as much between training seeds as between designs. Cheap simulations are the obvious screen, but whether they transfer to language-model agents is an empirical question. Surrogates fitted to an LLM's own decisions can reproduce the macro outcomes of LLM societies [4], and machine-learning surrogates can stand in for classic agent-based models [1]; across 260 configurations of real agent systems, a feature-based model picks the best architecture in most held-out cases [5]. We test a different and cheaper route: hand-built worlds with simple agents that are *not* fitted to any LLM, used to rank designs and to forecast the effect of reward designs before the expensive runs.

Our contributions: (1) pre-registered confirmatory tests of four hypotheses about shared information in a foraging world, all supported; (2) the transfer of design rankings to Claude Haiku 5.5 teams on three tasks, with one clear failure; (3) a forecast of RL reward designs for a 3B model, frozen before the evaluation and scored afterwards; (4) a characterisation of when the simulations transfer.

2 Simulations and their LLM counterparts

The foraging world. Agents move between resource patches on a ring or torus, share each patch's intake when they crowd it, and pay travel time; patches move, so agents must keep exploring. Each agent is a UCB

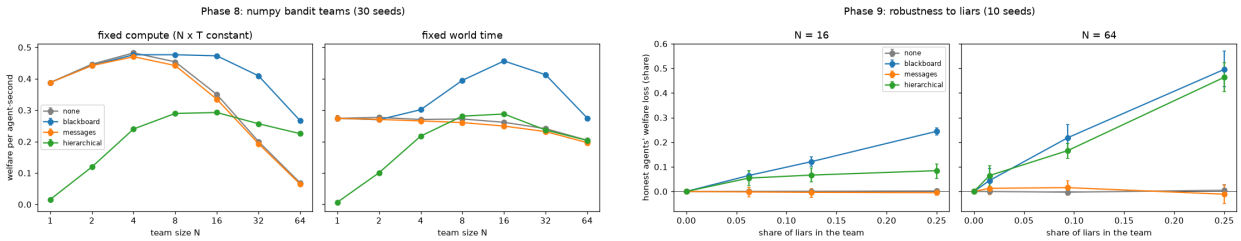


Figure 1: The simulation. Left: output per agent against team size under each channel (fixed compute and fixed world time). Right: honest agents' loss as the share of liars grows.

bandit over locations. Channels: none; a *blackboard* (a shared value map per location); *messages* (each agent sends a few location-value reports to peers); a *coordinator* that collects reports and assigns agents. Team sizes run from 1 to 64 under two budgets: fixed compute (total agent-seconds held constant, so larger teams run for less time each) and fixed world time. The reference is the ideal free distribution [3]. A scripted liar posts false patch values. Hypotheses and the analysis were frozen and hashed before the runs; each test used 30 seeds and Holm correction.

The task board and text foraging. The same channels were then given to Claude Haiku 5.5 teams of 1 to 8 agents on two text tasks: a task board where the risk is duplicate work, and foraging described in text. Numpy rule agents played the same tasks for the comparison.

The clue board. A task where information is the bottleneck: each subtask's answer is fixed by two modular clues dealt to different agents, so nobody can solve it alone and sharing costs a turn. Haiku teams and numpy rule agents played it under four channels and three team sizes.

Reward designs. A 3B open model (Qwen2.5-3B-Instruct), warm-started by imitating a rule agent, was trained with group-relative policy gradient on the clue board under five reward designs: individual, team, credit, individual with a training-only cost per wrong answer (`indpen`), and credit with per-turn advantages and the same cost (`credturn`). The numpy counterpart is the rule agent with two learned choices, whether to share a clue it could share and whether to guess on an undetermined subtask, trained by the same group-relative policy gradient from an imperfect imitation start, under the same five rewards, and evaluated with the LLM evaluation's metric definitions. Its method and scoring rule were committed before it was run, and its predictions for `indpen` and `credturn` before those runs were evaluated.

3 Results

3.1 Confirmatory tests in the simulation

Test (blackboard minus no channel; H3: minus messages)	Mean [95% CI]	Holm p	Verdict
P: welfare per agent-second, N 2–64, fixed compute	0.091 [0.088, 0.093]	0.0003	supported
H1: recovery time to the ideal free distribution (s)	−7.545 [−8.101, −7.030]	0.0003	supported
H2: belief diversity	−0.109 [−0.111, −0.108]	0.0003	supported
H3: honest agents' loss from one liar	0.041 [0.033, 0.049]	0.0001	supported

Table 1: Pre-registered tests in the numpy foraging world (30 seeds each, run once against the frozen plan).

A shared map keeps output per agent up as the team grows, speeds recovery after patches move, makes agents think alike, and makes the team more vulnerable to a liar than peer messages do (Table 1, Figure 1). The main grid ran in about 5 minutes on a laptop.

3.2 Do language-model teams rank designs the same way?

On the task board, a shared claim board kept welfare per agent-turn flat as Haiku teams grew, adding 0.60 $[0.57, 0.63]$ over no channel at eight agents, and the simulation ranked the 16 channel-size cells as the LLM teams did ($\rho = 0.75$ $[0.55, 0.93]$). On the clue board the agreement was as strong ($\rho = 0.79$ $[0.74, 0.81]$ over 12 cells); its main misses were the coordinator cells, where rule agents follow the coordinator and Haiku agents mostly ignored it. On text foraging the simulation failed ($\rho = 0.02$ $[-0.47, 0.45]$): the blackboard *cost* Haiku teams -0.34 $[-0.48, -0.13]$ at eight agents, with less exploration and lower belief diversity, the herding side of H2 [2] in a stronger form than the simulation showed. H3's direction did carry over: trained 3B agents lost more to a liar on the blackboard than with messages (0.311 $[0.189, 0.399]$ for the warm start, 0.500 $[0.280, 0.625]$ after RL with credit reward). For scale: the LLM main runs cost $32.68 for the task board and foraging study and $3.56 for the clue board; the simulations cost nothing beyond minutes of laptop time.

3.3 Forecasting reward designs

Scored with the frozen rule (the sign of each arm's effect against the imitation start on welfare, shares per held clue, solo share and wrong answers per solve; a numpy effect counts as + or − when its 95% interval over five training seeds excludes zero), the numpy learners reproduced 7 of 12 signs of the completed runs: all four for individual reward (sharing changed by -0.486 shares per held clue, guessing and wrong answers rose, welfare fell, as in the LLM run), three of four for team reward (missing the LLM run's collapse), none for credit reward. The forecast for the two unseen arms matched 4 of 8. It was right that RL with credit and per-turn advantages would beat the imitation start (numpy $+0.037$; LLM $+0.069$ $[+0.018, +0.120]$). It was wrong about the most interesting arm: it predicted that a training-only cost per wrong answer would not stop the selfish drift under individual reward (sharing -0.477, welfare -0.062), while the 3B model kept sharing ($+0.002$ $[-0.038, +0.042]$) and gained welfare ($+0.053$ $[+0.000, +0.106]$).

3.4 Forecasting team-size trends

Before running Claude Haiku teams of 8, 16 and 32 agents on the clue board, we froze a numpy forecast of two trends from 8 to 32 agents, scored with the same rule (a change counts as up or down when its interval, resampling seeds within each team size, excludes zero). The simulation predicted that the clean team's pooled share would fall (-0.156; observed -0.065 $[-0.081, -0.048]$) and that the honest agents' loss from a quarter of liars would fall too (-0.134), but it rose ($+0.076$ $[+0.013, +0.137]$): 1 of 2. We first guessed that rule agents dilute a liar's posts among more true ones. Tracing every turn of both runners (Haiku replayed from the response cache, a plan frozen first, no API calls) and splitting the honest loss into exactly additive terms ruled that out: lies were as visible at every size in both. The miss came from the clean trend. Rule teams pool so much worse as they grow (pooled share 0.51 to 0.36, against Haiku's 0.41 to 0.34) that liars have less to take: their loss on subtasks no lie touched fell from 0.37 to 0.23 per agent-turn, nearly all of their forecast fall, while their loss on poisoned subtasks stayed at 0.37. Haiku's untouched losses held (0.26 to 0.27) and its poisoned losses grew (0.25 to 0.31), as its agents built on a false clue more often (0.79 to 0.91 of answers). Three rule-agent variants, each copying one candidate mechanism (relaying seen clues, trusting the first sighting, reposting a true clue after a lie), all still predicted a fall (-0.23, -0.16, -0.05 against Haiku's $+0.08$; retrodictions): the error was in the clean baseline, not in how rule agents handle lies.

4 Discussion

When the simulation transfers. It transferred where the mechanism is information flow that rule agents and language models handle alike: duplicate work avoided by a claim board, clues that must reach whoever needs them, a false report spread by a shared board. It failed where language models do something rule agents cannot: ignoring a coordinator's suggestion, herding in free text, keeping more of their clean performance as teams grow (which leaves liars more to take), and, under RL, apparently learning that sharing speeds the team's solves even when the reward does not pay the sharer. In the reward forecast the two-decision learner cannot represent that last effect at all: under individual reward, sharing never pays it, whatever guessing

costs. A recent taxonomy of when fitted surrogates work, by what agents perceive and remember [4], offers a hypothesis to test: foraging in text gives LLM agents richer perception of others' behaviour than our bandits have.

A practical workflow. Use the simulation to rank channels and team sizes for information-sharing tasks, and to flag incentive problems that come from who gets paid (it predicted the selfish drift under individual reward exactly); do not use it to rank reward designs that work through what the model learns about the team, or for tasks where reading the shared text changes behaviour. Forecast an attack's effect and the clean trend it rests on separately, and check the clean trend first: a damage forecast inherits every error in the baseline it is measured against. When the simulation and a cheap LLM pilot disagree, the disagreement itself locates an LLM-specific behaviour worth studying.

Limitations. Three tasks with one LLM family for the rankings; one 3B model and two training seeds per reward design; rank agreement over 12–16 cells; a simple numpy learner for the reward forecast. The observed LLM signs in the reward forecast come from intervals over held-out boards pooled across two training seeds, and a later replication with four to five runs per arm found the loophole fix's effect on sharing seed-dependent; with seed-level variation as large as the welfare effects, the welfare signs in particular are less certain than the scores suggest. The frozen scoring rule gave small, tight numpy effects a definite sign; with a practical-significance threshold, reported post hoc, the scores are 6 of 12 and 5 of 8.

References

[1] Claudio Angione, Eric Silverman, and Elisabeth Yaneske. Using machine learning as a surrogate model for agent-based simulations. *PLoS ONE*, 17(2):e0263150, 2022.

[2] Sushil Bikhchandani, David Hirshleifer, and Ivo Welch. A theory of fads, fashion, custom, and cultural change as informational cascades. *Journal of Political Economy*, 100(5):992–1026, 1992.

[3] Stephen D. Fretwell and Henry L. Lucas. On territorial behavior and other factors influencing habitat distribution in birds. *Acta Biotheoretica*, 19:16–36, 1969.

[4] Y. Itkin. Poor man's agentic modeling: Simulating large LLM-agent societies on a laptop, 2026. arXiv:2608.11215.

[5] Yubin Kim et al. Towards a science of scaling agent systems, 2025. arXiv:2512.08296.

A Reproducibility

Code and notebooks: `https://github.com/Positron00/explRL`. Confirmatory plans: Phase 8 `PREREG` (hash `8a59293847ae`) and Phase 9 `PREREG_H3` (hash `a44617201f88`), frozen before their runs. The reward forecast: plan committed at `eb7b7c6`, predictions at `734839b`, both before the evaluated runs existed (`specs/multiagent/phase13/forecast.md`; `tools/rl_forecast.py`, byte-identical on rerun). The team-size forecast: `tools/scale_forecast.py`, committed before the Haiku runs; the traces that explain its miss: `TRACE_PLAN`, hash `f5c25637acd0`. Every number here is generated from the saved results by `paper2/make_assets.py`.